Министерство образования Республики Беларусь

Учреждение образования

«Брестский Государственный технический университет»

Кафедра ИИТ

Лабораторная работа №2

По дисциплине: «Конструирование программного обеспечения»

Тема: «Настройка взаимодействия с файлами»

Выполнила:

Студентка 2 курса

250851

Группы ПО-17

Бычик К.В.

Проверил:

Кулик А.Д.

Брест 2026

Цель работы: настроить взаимодействие между проектом, созданным в первой лабораторной работе, и файлами на компьютере. Добавить сохранение и загрузку настроек с использованием файлов, а также реализовать функциональность согласно индивидуальному варианту задания.

Ход работы

Вариант 2

2. Количество предметов хранится в переменной типа int, специальность и имя студента – в string. Все заданные имена студентов хранятся в отдельном файле, из которого они подгружаются в контейнер типа List. Все доступные специальности хранятся в отдельном файле, и из которого они при каждом запуске программы подгружаются в контейнер типа Set.

Код программы:

```
#define NOMINMAX

#include <iostream>
#include <string>
#include <fstream>
#include <list>
#include <set>
#include <windows.h>

using namespace std;

const int MENU_START = 1;
const int MENU_LOAD = 2;
const int MENU_SETTINGS = 3;
const int MENU_ABOUT = 4;
const int MENU_EXIT = 5;

const int SET_SPECIALITY = 1;
const int SET_NAME = 2;
const int SET_SUBJECTS = 3;
const int SET_BACK = 4;

const string VERSION = "2.0";

const string SETTINGS_FILE = "settings.txt";
const string STUDENTS_FILE = "students.txt";
const string SPECIALITY_FILE = "specialities.txt";

struct Settings {
    string speciality = "Программная инженерия";
    string studentName = "Кристина";
    int subjectsCount = 5;
};

void clearInput() {
    cin.clear();
    cin.ignore(10000, '\n');
}

void waitEnter() {
    cout << "\nНажмите Enter, чтобы вернуться в меню...";
    cin.get();
}

void ensureFile(const string& name, const string& content) {
    ifstream f(name);
    if (!f.good()) {
        ofstream(name) << content;
    }
}

void initFiles(const Settings& d) {
```

```
    ensureFile(SETTINGS_FILE,
        "speciality=" + d.speciality + "\n"
        "studentName=" + d.studentName + "\n"
        "subjectsCount=" + to_string(d.subjectsCount) + "\n");

    ensureFile(STUDENTS_FILE,
        "Кристина\nИван\nАнна\nПётр\nОльга\n");

    ensureFile(SPECIALITY_FILE,
        "Программная инженерия\n"
        "Информационные технологии\n"
        "Искусственный интеллект\n"
        "Кибербезопасность\n"
        "Веб-разработка\n");
}

void loadSettings(Settings& s) {
    ifstream in(SETTINGS_FILE);
    string line;
    while (getline(in, line)) {
        size_t p = line.find('=');
        if (p == string::npos) continue;
        string key = line.substr(0, p);
        string val = line.substr(p + 1);

        if (key == "speciality")        s.speciality = val;
        else if (key == "studentName")  s.studentName = val;
        else if (key == "subjectsCount") {
            try { s.subjectsCount = stoi(val); }
            catch (...) { s.subjectsCount = 5; }
        }
    }
}

void saveSettings(const Settings& s) {
    ofstream out(SETTINGS_FILE);
    out << "speciality=" << s.speciality << "\n"
        << "studentName=" << s.studentName << "\n"
        << "subjectsCount=" << s.subjectsCount << "\n";
}

template <typename Container>
void loadLines(const string& file, Container& c) {
    c.clear();
    ifstream in(file);
    string line;
    while (getline(in, line)) {
        if (!line.empty()) c.insert(c.end(), line);
    }
}

template <typename Container>
bool chooseFrom(const Container& c, string& out) {
    if (c.empty()) { cout << "Список пуст.\n"; return false; }

    int i = 1;
    for (const auto& v : c) cout << "  " << i++ << ". " << v << "\n";
    cout << "Выберите номер (0 - отмена): ";

    int n;
    cin >> n;
    if (cin.fail() || n < 0 || n >(int)c.size()) {
        clearInput();
        cout << "Некорректный выбор!\n";
        return false;
    }
    if (n == 0) return false;

    auto it = c.begin();
    advance(it, n - 1);
    out = *it;
    return true;
}

void startGame() {
```

```
    cout << "\n=== Начало игры \"Моя жизнь в университете\" ===\n";
    cout << "Вы поступаете в университет. Удачи!\n";
    waitEnter();
}

void loadGame() {
    cout << "\nФункция загрузки пока недоступна.\n";
    cout << "Она появится в ближайших обновлениях.\n";
    waitEnter();
}

void aboutProgram() {
    cout << "\n=== О программе ===\n";
    cout << "Версия: " << VERSION << "\n";
    cout << "Студент: Бычик К.В., группа ПО-17\n";
    waitEnter();
}

void settingsMenu(Settings& s,
    const list<string>& students,
    const set<string>& specialities) {
    int choice = 0;
    while (choice != SET_BACK) {
        cout << "\n--- Настройки ---\n";
        cout << "1. Специальность (текущая: " << s.speciality << ")\n";
        cout << "2. Имя студента (текущее: " << s.studentName << ")\n";
        cout << "3. Количество предметов (текущее: " << s.subjectsCount << ")\n";
        cout << "4. Назад\n";
        cout << "Выбор: ";

        cin >> choice;
        if (cin.fail()) {
            clearInput();
            cout << "Некорректный ввод!\n";
            continue;
        }

        switch (choice) {
        case SET_SPECIALITY:
            clearInput();
            cout << "\nДоступные специальности:\n";
            if (chooseFrom(specialities, s.speciality)) {
                saveSettings(s);   // сохраняем сразу
            }
            break;

        case SET_NAME:
            clearInput();
            cout << "\nДоступные студенты:\n";
            if (chooseFrom(students, s.studentName)) {
                saveSettings(s);
            }
            break;

        case SET_SUBJECTS: {
            cout << "Введите количество предметов: ";
            int n;
            cin >> n;
            if (cin.fail() || n <= 0) {
                clearInput();
                cout << "Некорректное значение!\n";
            }
            else {
                s.subjectsCount = n;
                saveSettings(s);
            }
            break;
        }

        case SET_BACK:
            break;

        default:
            cout << "Такого пункта нет.\n";
        }
```

```
    }
}

int main() {
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);
    setlocale(LC_ALL, "Russian");

    Settings settings;

    // НОВОЕ: файлы и контейнеры
    initFiles(settings);
    loadSettings(settings);

    list<string> students;
    set<string> specialities;
    loadLines(STUDENTS_FILE, students);
    loadLines(SPECIALITY_FILE, specialities);

    int choice = 0;
    while (choice != MENU_EXIT) {
        cout << "\n*** Моя жизнь в университете ***\n";
        cout << "1. Начать игру\n";
        cout << "2. Загрузить игру\n";
        cout << "3. Настройки\n";
        cout << "4. О программе\n";
        cout << "5. Выход\n";
        cout << "Выбор: ";

        cin >> choice;
        if (cin.fail()) {
            clearInput();
            cout << "Некорректный ввод!\n";
            continue;
        }

        switch (choice) {
        case MENU_START:    startGame(); break;
        case MENU_LOAD:     loadGame();  break;
        case MENU_SETTINGS: settingsMenu(settings, students, specialities); break;
        case MENU_ABOUT:    aboutProgram(); break;
        case MENU_EXIT:
            cout << "Выход из программы...\n";
            break;
        default:
            cout << "Такого пункта нет.\n";
        }
    }

    return 0;
}
```

Результат программы:

```
*** Моя жизнь в университете ***
1. Начать игру
2. Загрузить игру
3. Настройки
4. О программе
5. Выход
Выбор: 1

=== Начало игры "Моя жизнь в университете" ===
Вы поступаете в университет. Удачи!
*** Моя жизнь в университете ***
1. Начать игру
2. Загрузить игру
3. Настройки
4. О программе
5. Выход
Выбор: 3

--- Настройки ---
1. Специальность (текущая: Программная инженерия)
2. Имя студента (текущее: Кристина)
3. Количество предметов (текущее: 5)
4. Назад
Выбор:
```

```
*** Моя жизнь в университете ***
1. Начать игру
2. Загрузить игру
3. Настройки
4. О программе
5. Выход
Выбор: 2

Функция загрузки пока недоступна.
Она появится в ближайших обновлениях.
```

```
--- Настройки ---
1. Специальность (текущая: Программная инженерия)
2. Имя студента (текущее: Кристина)
3. Количество предметов (текущее: 5)
4. Назад
Выбор: 3
Введите количество предметов: 11

--- Настройки ---
1. Специальность (текущая: Программная инженерия)
2. Имя студента (текущее: Кристина)
3. Количество предметов (текущее: 11)
4. Назад
Выбор:
```

```
*** Моя жизнь в университете ***
1. Начать игру
2. Загрузить игру
3. Настройки
4. О программе
5. Выход
Выбор: 4

=== О программе ===
Версия: 2.0
Студент: Бычик К.В., группа ПО-17

Нажмите Enter, чтобы вернуться в меню...
*** Моя жизнь в университете ***
1. Начать игру
2. Загрузить игру
3. Настройки
4. О программе
5. Выход
Выбор: 5
Выход из программы...
```

Контрольные вопросы:

0. Алгоритм работы

1. Настройка кодировки консоли (CP1251).
   Создание Settings со значениями по умолчанию.

   initFiles() - создаёт settings.txt, students.txt, specialities.txt, если их нет.

   loadSettings() - читает настройки из settings.txt.

   loadLines() - грузит студентов в list, специальности в set.

   Главный цикл меню: 1 - игра, 2 - загрузка, 3 - настройки, 4 - о программе, 5 - выход.

   В настройках: выбор специальности из set, имени из list, ввод количества предметов. После каждого изменения - saveSettings().

2. При выходе настройки уже сохранены - при следующем запуске читаются из файла.

1. Библиотеки файлового ввода-вывода

<fstream> - три класса:
ifstream - чтение,
ofstream - запись,
fstream - и то, и другое.

Методы: open(), close(), is_open(), good(), getline(), <<, >>.
Режимы: ios::in, ios::out, ios::app, ios::trunc, ios::binary.

2. Контейнеры

Шаблонные классы STL для хранения данных:

- **последовательные:** vector, list, deque;
- **ассоциативные:** set, map;
- **адаптеры:** stack, queue.

В работе:

- list<string> - двусвязный список, быстрая вставка/удаление, нет доступа по индексу. Для студентов.

- set<string> - упорядоченное множество уникальных элементов, авт. сортировка, поиск O(log n). Для специальностей.

3. Логирование

Это запись событий программы (запуск, действия пользователя, ошибки) в отдельный файл - **лог** - для отладки и анализа. Зачем нужно: восстановить хронологию при сбое, найти ошибку, собрать статистику.

Есть уровни важности: **DEBUG** (отладка), **INFO** (штатные события), **WARNING** (предупреждения), **ERROR** (ошибки), **FATAL** (критические сбои).

В C++ пишут либо вручную через ofstream в режиме дозаписи, либо используют готовые библиотеки: **spdlog**, **Boost.Log**, **glog**.

4. Unit-тестирование

Это автоматическая проверка отдельных функций (юнитов): подаём известные входные данные - сверяем результат с ожидаемым. Зачем: ловить ошибки при изменении кода, упрощать рефакторинг, документировать поведение функций.

Популярные фреймворки для C++: **Google Test**, **Catch2**, **doctest**, **Boost.Test**.

Тест обычно состоит из трёх шагов: подготовка данных → вызов функции → сравнение результата с эталоном. Если не совпало - тест падает. Плюс - быстрая проверка, минус - нужно время на написание и поддержку.

5. Обработка исключений

Это механизм C++ для реакции на ошибки без аварийного завершения программы. Состоит из трёх частей: throw - сгенерировать исключение, try - блок с «опасным» кодом, catch - блок обработки.

Стандартные исключения: invalid_argument (неверный аргумент), out_of_range (выход за границы), runtime_error (ошибка во время работы), bad_alloc (нехватка памяти).

В нашей программе исключения нужны при чтении числа из файла: если вместо числа окажется мусор, stoi бросит исключение, и без обработки программа упала бы. С try/catch подставляется значение по умолчанию.

Правила: ловить по константной ссылке, не глотать исключения молча, обрабатывать там, где есть информация для реакции.

Вывод: в ходе лабораторной работы программа была доработана: сохранена вся функциональность предыдущей лабораторной работы, при этом добавлены новые возможности и усложнена структура проекта за счёт взаимодействия с файлами.